

Kosmocrates

Coffin-Dragger & Wings of Samael

Closed Master Specification & Implementation Architecture

A rigorous, repository-grounded synthesis of the
ACDC / CDK / ASCC attractor calculus, the Wings-of-Samael radial projection layer,
MERKABA runtime geometry, the Metatron Projection Kernel, QSR stabilization, and
DiamondCube condensation,
prepared for direct hand-off to an implementation agent.

Document class	Closed master specification (implementation-oriented)
Specification version	v1.0-closed
Stage identity	ASCC fold of the Coffin-Dragger corpus
Source corpus author	Sebastian Klemm
Target executor	Claude Code (repository implementation agent)
Engine	pdfL ^A T _E X (Overleaf compatible)

This document makes no claim of physical realization of Quantenstarre, gravastars, time-bridges, or mycelial structures. All such terms are read strictly as operator-, boundary-, fixpoint-, projection-, or control structures. Pictorial origin terms survive only as internal codenames; the normative text uses formal roles.

Contents

Executive Summary	4
I Doctrine and Invariant Order	5
1 Status, Scope, and Claim Boundaries	6
1.1 Position in the stack	6
1.2 Non-goals	6
1.3 Claim boundaries	6
2 Source Families and Crystallized Invariants	7
2.1 Bearing source families	7
2.2 The seven core invariants	8
3 Overall Thesis	9
3.1 Formal thesis	9
3.2 Operational thesis	9
3.3 Consolidated target frame	9
4 Layer Model	10
4.1 Normative layering	10
4.2 No short-circuit	10
II Formal Core: Objects, Operators, Calculi	11
5 Ground Objects	12
5.1 State	12
5.2 Canonicalization and semantic equivalence	12
5.3 Stage	12
5.4 Attractor stack and stage delta	13
6 Operator Algebra	14
6.1 Operator space	14
6.2 Reflection, inversion, seam (ANL/SRG)	14
6.3 Exkalibration	14
7 Quantenstarre and the QSR Predicate	15
7.1 Stage-level QSR	15
7.2 Stack-level QSR	15
7.3 QSR as condensation guard	15

8	Attractor Stack Closure Calculus (ASCC)	16
8.1	Canonical stage embedding	16
8.2	Support-preserving accretion	16
8.3	Contractive consolidation	16
8.4	Stack closure	16
8.5	Closure schema (master recurrence)	17
9	ACDC: Centralization / Decentralization	18
9.1	Adjunction	18
9.2	Roundtrip stability	18
10	CDK: The Coffin-Dragger Cycle	19
10.1	Run as stage transition	19
10.2	Peristaltic motion	19
10.3	Arm outputs as gated deltas	19
III	Architecture: Wings, Ophanim, Oracle, Runtime, Materialization	20
11	Wings of Samael: Radial Arm Projection Layer (L3)	21
11.1	Role	21
11.2	Wing operator path	21
11.3	Wing invariant	21
11.4	Wing types	21
12	Wing Algebra and Ophanim Topology	23
12.1	Compositions	23
12.2	Wing mesh	23
12.3	Ophanim cycle	23
13	Metatron Projection Kernel (L4)	24
13.1	Position	24
13.2	Function	24
14	MERKABA Runtime Geometry (L6)	25
14.1	Role	25
14.2	Oracle discipline	25
15	Panoptic Projection (L5)	26
16	HYPHAE, LPCM, SystemCube (L8 substrate)	27
16.1	HYPHAE	27
16.2	LPCM	27
16.3	SystemCube	27
17	DiamondCube Condensation (L7→L8)	28
17.1	Definition	28
17.2	Strong irreducibility	28
17.3	Diamondization	28
18	Metrics and the <i>D</i>-Energy Kernel	29
18.1	Tripolar epistemic energy	29
18.2	Contradiction energy and the objective	29
18.3	Stack and cube metrics	29

18.4 Stage progress	29
IV Implementation Specification (Repository Hand-off)	30
19 Kosmocrates Binding Layer (KBL)	31
19.1 Purpose	31
19.2 Binding invariant	31
20 Algorithms	32
20.1 Attractor Stack Closure run	32
20.2 Recursive assimilation stage (CDK inner cycle)	32
20.3 Diamondization run	32
21 Data Models and JSON Schemas	34
22 Rust Type and Trait Surface	36
23 Module / Crate Layout	38
24 API and CLI	39
24.1 CLI verbs	39
24.2 REST endpoints (mirror of CLI)	39
25 Gate and Safety Discipline	40
25.1 Mandatory gate cascade	40
25.2 Non-negotiable rule	40
26 Negative Tests and Conformance	41
26.1 Negative tests (must be detected and fail-closed)	41
26.2 Conformance classes	41
26.3 Definition of Done	42
27 Repository-Agent Work Plan	43
27.1 Phase 1 — Inventory	43
27.2 Phase 2 — Mapping	43
27.3 Phase 3 — Specification	43
27.4 Phase 4 — Implementation tasks	43
27.5 Success criteria	43
28 Direct Hand-off Prompt for Claude Code	45
Closing	47
Closing Formula	47
A Codename and Formal-Name Mapping	48
B Terminology Cleanup	49
C Source Lineage	50

Executive Summary

This specification closes the Coffin-Dragger corpus. It takes the recursively elaborated stack of source documents—PSP state-first semantics, the ANL/SRG reflection boundary, Quantenstarre/QSR stability, the Metatron Projection Kernel (MPK), HYPHAE/LPCM/SystemCube topological accretion, the ACDC/CDK attractor calculus, and the Attractor Stack Closure Calculus (ASCC)—and folds it into a single, canonical, content-addressed, evidence-bound architecture that an implementation agent can realize as a set of Rust crates and a CLI inside the Kosmocrates repository.

Governing thesis. *Coffin-Dragger does not copy systems. It accretes their topology, condenses their invariants, certifies the condensation under QSR, and materializes only what the host can survive.* Formally,

$$\text{CoffinDragger} = \text{Diamondize}\left(\text{Core}\left(\text{Fix}\left(\Omega(\text{WingsOfSamael}(\partial\text{QSR}))\right)\right)\right),$$

and operationally, the Coffin-Dragger Kalkül (CDK) is the bidirectional cycle

$$\text{CDK} = \text{Cen} \circ \text{Probe} \circ \text{Seed} \circ \text{Purge} \circ \text{Pull}.$$

What the implementation agent receives. A layered architecture (L0–L8) with no permitted short-circuit; a normative object model (Stage, AttractorStack, Wing, Ophanim, DiamondCube); the closed ASCC fold calculus with embedding and residue discipline; a complete gate matrix where *scores rank and gates decide*; JSON schemas and Rust trait signatures; a crate layout (kosmo-cdk-core, kosmo-wings, kosmo-coffindragger, tools/kosmo-cdk); a report-only MVP path with a staged escalation to operator-approved materialization; and a full negative-test and acceptance matrix that defines done.

Energy basis. All ranking reuses the Kosmocrates tripolar epistemic energy $D = \psi \cdot \rho \cdot \omega$ (meaning $\times$ coherence $\times$ phase), penalized by contradiction energy K and avoidable complexity C . A good condensate is never the largest; it is the one of maximal target-compatible D -density under minimal contradiction energy, certified irreducible by QSR.

Part I

Doctrine and Invariant Order

Chapter 1

Status, Scope, and Claim Boundaries

1.1 Position in the stack

This document is the *closure stage* of the Coffin-Dragger corpus. Earlier stages defined the constituent calculi independently. The Master Handoff Package (v0.9) assembled them into a coherent frame but explicitly declined to invent repository detail. The Attractor Stack Closure Calculus (ASCC, v1.3) supplied the missing finiteness/abstraction: a recursively growing stack of stages must be *folded* into a finite, auditable, content-addressed bundle rather than accumulate loose layers. The present document performs that fold over the whole corpus and emits one implementation-grade specification.

Formally this document is itself a stage S_N satisfying the ASCC embedding law

$$\text{Canon}(\text{Embed}_n(S_n)) = \text{Canon}(S_n), \quad \text{Support}(S_n) \setminus \text{Support}(S_{n+1}) \subseteq R_n,$$

i.e. it preserves the canonical substance of every source stage and records any removed material as explicit residue (chapter 8).

1.2 Non-goals

This specification is *not*: a replacement for code, tests, domain experts, or build/validation; a claim that any generated artifact is absolutely perfect; a universal translator without adapters; a license-bypass or raw-code ingestion format; a model-training dataset format; an autonomous authority surface; nor a reason to bypass fail-closed governance. It claims no physical realization of any pictorial source term.

1.3 Claim boundaries

- All evocative names (Wings of Samael, Ophanim, Gravastar axis, Metatron cube, Quantenstarre, wormhole, mycelium) are *codenames*. The normative text uses their formal roles (radial projection arm, closed wing cycle, QSR-stable boundary invariance, finite symmetry oracle, structural invariance predicate, identification/remapping relation, pull/harvest infrastructure).
- “Centralization/decentralization,” “cloning,” and “traversal” denote fold/unfold, branch-projection, and identification operators respectively, never physical processes.
- Intelligence in this architecture arises from control structure—gates, synchronization, memory, boundary, and replay—not from any single generator. An LLM is an *oracle*, never the authority.

Chapter 2

Source Families and Crystallized Invariants

2.1 Bearing source families

Family	Invariant contribution
PSP / state-first	State-first semantics, typed multigraph state, operator calculus, deterministic canonicalization, gates, replayable evidence.
ANL / SRG	Autoreflexive null-lens; self-adjoint involutive reflection boundary; stateless null-anchor; mirror-seam and boundary discipline; non-traversal self-identification.
Quantenstarre / QSR	Maximal structural invariance of a state core under perturbation, reflection, inversion, projection, gating, and trace-replay; exkalkibration as active boundary ejection.
Kaskaden-Tunnel	Dual split / mirror / inversion operators; depth cascade onto a shared null-center; tunnel boundary as quotient/rigidity limit.
DNK	Domain-neutral crystal: non-singular condensation, inverted boundary membrane, hypercube compression to an irreducible invariant system.
MPK / Metatron-scan	Finite, deterministic graph/symmetry oracle for $n \leq 8$; canonicalization, orbit, stabilizer, spectrum, complement, catalog identity; the 13598-graph catalog as a finite structure space.
TPC / PSE Traverse	Panoptic orientation; phase horizons; attractor maps; gated emission; navigation of high-dimensional possibility spaces.
HYPHAE / LPCM / SystemCube	Host-bound, evidence-first, non-copying topology assimilation; controlled fragmentation; monotone contractive reduction of spurious degrees of freedom; exportable content-addressed topological blueprint.
ACDC / CDK	Bidirectional attractor centralization/decentralization; the practical Pull/Purge/Seed/Probe/Cen runtime cycle.
ASCC / MRAC	Recursive stage embedding, support-preserving accretion, contractive consolidation, stack closure, diamondization.
PhaseMatrix / Hivemind	Federated local phase-space systems; mirror-seams; cross-certification; lattice certificates.

2.2 The seven core invariants

Every layer, type, and gate below is a refinement of one of these. They are the fixed substance the fold must preserve.

- I1. Structure over surface.** Code, text, and reports are carriers; only their canonicalizable topology is relevant.
- I2. Gate before score.** Metrics rank; gates permit or block. A higher score never overrides a gate.
- I3. Boundary over absorption.** Non-integrable structure is made visibly exkalibrated, never silently swallowed.
- I4. Trace and replay.** Every relevant artifact is deterministically reproducible or explicitly marked incomplete.
- I5. Fixpoint over target text.** Convergence is to a stable attractor A^* , not to a single output.
- I6. Projection over omniscience.** Panoptic sight is a bounded canonical map of reachable / latent / blocked regions.
- I7. Condensation over growth.** Increasing capability must coincide with increasing density, purity, or irreducibility.

Chapter 3

Overall Thesis

3.1 Formal thesis

$$\text{CoffinDragger} = \text{Diamondize}\left(\text{Core}\left(\text{Fix}\left(\Omega(\text{WingsOfSamael}(\partial\text{QSR}))\right)\right)\right). \quad (3.1)$$

In words: Coffin-Dragger emits radial projection arms out of a QSR-stable boundary, couples them into Ophanim cycles, audits them through MPK / TPC / MERKABA, takes the irreducible core of the stable cycles, and condenses it into a DiamondCube.

3.2 Operational thesis

$$\text{CDK} = \text{Cen} \circ \text{Probe} \circ \text{Seed} \circ \text{Purge} \circ \text{Pull}, \quad A_{n+1} = \text{Pull}(\text{Probe}(\text{Seed}(\text{Purge}(A_n)))). \quad (3.2)$$

Pull

draw structure from domains and corpora toward the attractor.

Purge

excalibrate foreign mass; make residue visible.

Seed

expose the cleaned core as sub-attractors, scaffolds, probes.

Probe

validate the seeded sub-attractors.

Cen

re-condense feedback and new evidence into the attractor.

3.3 Consolidated target frame

$$\text{DiamondCubeCandidate} = \text{KBL}\left(\text{ASCC}(\text{CDK}(\text{HYPHAE}, \text{LPCM}, \text{MPK}, \text{TPC}, \text{MERKABA}))\right). \quad (3.3)$$

This is the target the implementation realizes: a Kosmocrates Binding Layer wrapping an ASCC closure over a CDK run that orchestrates the assimilation, contraction, symmetry, projection, and runtime subsystems.

Chapter 4

Layer Model

4.1 Normative layering

	Layer	Role
L0	Null-anchor	Stateless order anchor; no data-bearing vertex.
L1	Quantenstarre / QSR	Stability and closure predicate for attractor cores.
L2	ANL / SRG	Reflection boundary, mirror-seam, boundary discipline.
L3	Wings of Samael	Radial projection-arm layer between boundary and scan space.
L4	MPK cluster	Local finite symmetry / graph oracle inside wings.
L5	Panoptic projection	Orbit completion, attractor maps, phase horizons (TPC).
L6	MERKABA runtime	Scheduler, synchronizer, gatekeeper, Ophanim orchestration.
L7	Coffin-Dragger / CDK	Global attractor centralization and diamondization.
L8	Materialization	SystemCube / WishCube / Foundry / PSE bridge / repo output.

4.2 No short-circuit

Requirement 4.1 (Layer traversal). An implementation **MUST NOT** route a materializable effect directly from L1 to L8. Every materializable effect **MUST** pass boundary (L2), projection arm (L3), scan/projection (L4–L5), runtime gate (L6), stack closure and global centralization (L7), and QSR before reaching materialization (L8). This prevents symbolic overload and keeps the architecture auditable.

Part II

Formal Core: Objects, Operators, Calculi

Chapter 5

Ground Objects

5.1 State

Definition 5.1 (PSP-extended state). A state is the typed tuple

$$\Sigma = (G, z, \Pi, \mu, H, RD; C, F, B, P),$$

where (G, z, Π, μ, H, RD) is the PSP core (typed multigraph G , canonical serialization z , projection map Π , measure μ , history H , run descriptor RD) and the Quantenstarre extension adds the constraint lattice component C , the foreign/noise register F , the boundary/seam register B , and the projection/orbit register P .

Axiom 5.2 (State-first). *Every semantically relevant operation is an operator application on Σ or a gate/trace object derived from such an application.*

Axiom 5.3 (Boundary, not absorption). *Non-integrable foreign mass MUST NOT silently enter the core. It is exkalibrated, quarantined, or logged as a gate error.*

Axiom 5.4 (Null-anchor statelessness). *The null-anchor N_0 is not a data-bearing state, vertex, store, or dynamic object. It appears only as an order reference in operator and gate definitions.*

5.2 Canonicalization and semantic equivalence

Canonicalization is deterministic and run-descriptor bound. Two states are equivalent iff their canonical forms coincide:

$$q \equiv q' \iff \text{Canon}(q) = \text{Canon}(q').$$

$\text{Sem}(\cdot)$ denotes the semantic load of a unit; $\simeq$ denotes equivalence under the accepted canonicalization and gate system (not surface identity).

5.3 Stage

Definition 5.5 (Stage). A stage is the tuple

$$S_n = (A_n, U_n, E_n, G_n, T_n, R_n, M_n, C_n),$$

with attractor state A_n , semantically bearing units U_n , evidence store E_n , gate status G_n , trace/replay data T_n , residue R_n , metrics M_n , certificates C_n .

5.4 Attractor stack and stage delta

Definition 5.6 (Attractor stack). An attractor stack is a finite (or incrementally growing) sequence $\text{Stack}_N = (S_0, \dots, S_N)$ together with embeddings $\text{Embed}_n : S_n \hookrightarrow S_{n+1}$. It is not a mere history: it is the formal carrier of the claim that each successor stage preserves the canonical substance of its predecessor.

Definition 5.7 (Stage delta). The transition $S_n \rightarrow S_{n+1}$ is described by $\Delta_n = (I_n, F_n, X_n, Z_n)$: invariants I_n , new fragments F_n , exclusions X_n , certificates Z_n . The delta is itself evidence-bound and replayable.

Chapter 6

Operator Algebra

6.1 Operator space

The operator space is the typed union

$$\Omega = \Omega_{\text{base}} \cup \Omega_{\text{bdry}} \cup \Omega_{\text{proj}} \cup \Omega_{\text{excal}} \cup \Omega_{\text{gate}}.$$

Every operator carries type, domain, codomain, an admissibility predicate, and a trace schema. An operator application is admissible only if its predicate holds on the canonical input.

6.2 Reflection, inversion, seam (ANL/SRG)

Definition 6.1 (Self-adjoint reflection boundary). Let R be an involutive reflection ($R \circ R = \text{id}$) and J an inversion. The observable boundary is the fixed set

$$\partial Q = \text{Fix}(R) = \{ q \mid R(q) = q \}.$$

The two “sides” of a structure are not joined by a path; they are recognized as inversions J of one structure. The seam operator Q_s records the mirror-seam where reflection meets inversion. The null-anchor N_0 is the stateless order reference of R .

Contract 6.2 (Non-traversal). No admissible operator realizes a path between the two sides of ∂Q . Apparent connection is identification under R , certified by Q_s , never traversal.

6.3 Exkalibration

Definition 6.3 (Exkalibration family). $\text{ExCal} \subseteq \Omega_{\text{excal}}$ is the family that ejects foreign mass, noise, and incompatible constraints to the boundary, contractively condenses the remaining core, and projects it into a QSR-invariant region. Every ejection produces a boundary certificate $b \in B_Q$ documenting what was separated. Constraints form a lattice $C = (C, \leq, \wedge, \vee)$; incompatible constraints are referred to the boundary, never merged.

Chapter 7

Quantenstarre and the QSR Predicate

7.1 Stage-level QSR

Definition 7.1 (Stage-QSR). $\text{QSR}_{\text{stage}}(S_n) = 1$ iff all of:

$\text{Gate}(G_n) = 1$, $\text{Replay}(S_n) = 1$, $\text{Canon}(S_n)$ stable, $\text{Irr}(S_n)$ high or rising, $\text{Contr}(S_n) \leq \text{Contr}(S_{n-1})$.

7.2 Stack-level QSR

Definition 7.2 (Stack-QSR).

$$\text{QSR}_{\text{stack}} = 1 \iff (\forall n : \text{QSR}_{\text{stage}}(S_n) = 1) \wedge \text{Canon}(S_n) \rightarrow A^\star.$$

The stack is stable when every accepted stage passes stage-QSR and the roundtrip returns to the same attractor.

7.3 QSR as condensation guard

Invariant 7.3 (QSR monotonicity). *Across accepted stages, density and purity are non-decreasing and contradiction energy is non-increasing:*

$$\text{Density}(S_{n+1}) \geq \text{Density}(S_n), \quad \text{Purity}(S_{n+1}) \geq \text{Purity}(S_n), \quad K(S_{n+1}) \leq K(S_n).$$

Violation without an explicit gate-justified exception fails QSR.

Chapter 8

Attractor Stack Closure Calculus (ASCC)

8.1 Canonical stage embedding

Definition 8.1 (Valid embedding). $\text{Embed}_n : S_n \hookrightarrow S_{n+1}$ is valid iff $\text{Canon}(\text{Embed}_n(S_n)) = \text{Canon}(S_n)$ and $\text{Sem}(\text{Embed}_n(S_n)) \simeq \text{Sem}(S_n)$. It may change representation, indexing, layout, or implementation detail, but not canonical structure. Each embedding emits a certificate $\text{Cert}_n^{\text{embed}} = (\text{id}_n, S_n, S_{n+1}, \text{Canon}, \text{Gate}, \text{Trace}, \text{Replay})$. Without this certificate the successor is merely a new version, not a valid assimilation stage.

8.2 Support-preserving accretion

Definition 8.2 (Support). $\text{Support}(S_n) = \{u \in U_n \mid \text{Gate}(u) = 1 \wedge \text{Sem}(u) \neq \emptyset\}$.

Definition 8.3 (Accretion rule).

$$\text{Support}(S_{n+1}) = (\text{Support}(S_n) \setminus X_n) \cup A_n^+,$$

with explicit exclusion X_n and admitted growth A_n^+ .

Contract 8.4 (No covert removal). $\text{Support}(S_n) \setminus \text{Support}(S_{n+1}) \subseteq X_n \cup R_n$. Any unit that leaves the support MUST appear in the exclusion set or the residue. Cleaning is never silent forgetting.

8.3 Contractive consolidation

Definition 8.5 (Contraction). $\text{Contr} : S_n \rightarrow S'_n$ reduces cost, redundancy, and contradiction: $\text{Cost}(S'_n) \leq \text{Cost}(S_n)$. It is valid only if semantic load is preserved ($\text{Sem}(S'_n) \simeq \text{Sem}(S_n)$) or lost portions are explicitly recorded as residue.

Definition 8.6 (Local irreducibility). S_n is locally irreducible iff no further contraction is possible without losing semantics, gate status, or materializability:

$$\forall c : \text{Contr}_c(S_n) \neq S_n \Rightarrow \neg(\text{Sem}(\text{Contr}_c(S_n)) \simeq \text{Sem}(S_n)).$$

8.4 Stack closure

Definition 8.7 (Fold). $\text{Fold}_N : \text{Stack}_N \rightarrow B_N$, where B_N is not a text summary but a content-addressed, evidence-bound structural bundle.

Definition 8.8 (Closure condition). A stack is closed iff $\text{Gate}(B_N) = 1$, $\text{Replay}(B_N) = 1$, and $\text{Canon}(B_N) = \text{Canon}(S_N)$.

Proposition 8.9 (Stability under further fold). *The stack is stable iff $\text{Fold}_{N+1}(\text{Stack}_N, S_N) \equiv \text{Fold}_N(\text{Stack}_N)$, i.e. further folding produces no new canonical structure—recursive elaboration yields only certified condensation, not new ballast.*

8.5 Closure schema (master recurrence)

$$S_{n+1} = \text{Recryst} \left(\text{Contr} \left(\text{Accrete}(\text{Embed}_n(S_n), \Delta_n) \right) \right), \quad (8.1)$$

subject to

$$\text{Canon}(\text{Embed}_n(S_n)) = \text{Canon}(S_n), \quad \text{Support}(S_n) \setminus \text{Support}(S_{n+1}) \subseteq R_n, \quad \text{QSR}(\text{Press}_{\diamond}(\text{Fold}(\text{Stack}_N))) = 1.$$

Chapter 9

ACDC: Centralization / Decentralization

9.1 Adjunction

ACDC is the research calculus for attractor centralization Cen and decentralization Dec , with $\text{Cen} \dashv \text{Dec}$. Centralization is a fold of a field into an attractor stack, $\text{Cen}(\mathcal{X}) = \text{Fold}(\text{Stack}(\mathcal{X}))$; decentralization produces gate-bound sub-attractors, $\text{Dec}(A) = \{a_i \mid \text{Gate}(a_i) = 1 \wedge \text{Trace}(a_i) = 1\}$, never random expansion.

9.2 Roundtrip stability

Definition 9.1 (ACDC roundtrip). The roundtrip is stable iff $\text{Cen}(\text{Dec}(A^*)) \equiv A^*$. Stack closure provides the checkable form

$$\text{Canon}(\text{Fold}(\text{Stack}(\text{Dec}(A^*)))) = \text{Canon}(A^*).$$

Chapter 10

CDK: The Coffin-Dragger Cycle

10.1 Run as stage transition

A CDK run produces not just an output but a new stage: $\text{Run}_n : (S_n, RD_n) \rightarrow S_{n+1}$. The run is valid iff it logs embedding, accretion, contraction, and fold.

10.2 Peristaltic motion

The runtime body advances as $\text{Front}_n \rightarrow \text{Anchor}_n \rightarrow \text{PullBody}_n \rightarrow \text{Compress}_n$, i.e. $S_{n+1} = \text{Compress}(\text{Pull}(\text{Embed}(S_n), \Delta_n))$: the new stage reaches forward (new idea/module/link), anchors the front to A^* , pulls the prior stages along, then canonicalizes and condenses the whole body.

10.3 Arm outputs as gated deltas

Each wing (arm) H_i produces a delta $H_i(D_i) = \Delta_{n,i}$. The stage delta is the gate-checked sum

$$\Delta_n = \bigoplus_i \text{Gate}(\Delta_{n,i}),$$

where $\oplus$ is defined only for compatible, boundary-conforming, replayable deltas. Conflicting arms are not summed; the conflict is written to residue/boundary.

Part III

Architecture: Wings, Ophanim, Oracle, Runtime, Materialization

Chapter 11

Wings of Samael: Radial Arm Projection Layer (L3)

11.1 Role

WingsOfSamael is a technical intermediate membrane, not a poetic figure:

$$\text{WingsOfSamael} : \partial Q \rightarrow \{w_1, \dots, w_n\}.$$

It transforms a fixed-axis / boundary structure into scannable, gate-bound projection arms.

11.2 Wing operator path

Definition 11.1 (Wing). A single wing is the operator path

$$w_i = \text{Anchor}_i \circ \text{Condense}_i \circ \text{Harvest}_i \circ \text{Orbit}_i \circ \text{Probe}_i,$$

where *Probe* spans a local observation horizon, *Orbit* generates admissible variant/structure classes, *Harvest* pulls back only invariant-relevant structure, *Condense* reduces degrees of freedom and surface noise, and *Anchor* binds the result to the target attractor.

11.3 Wing invariant

Invariant 11.2 (Wing condensation). *Every admissible wing reduces degrees of freedom or isolates their relevance with justification:*

$$\dim(w_i(x)) \leq \dim(x) \quad \text{or} \quad \dim_{\text{relevant}}(w_i(x)) < \dim_{\text{relevant}}(x).$$

Exploration may enlarge the raw landscape, but the accepted wing output MUST condense.

11.4 Wing types

Type	Task
ScoutWing	Open new domain horizons; do not integrate directly.
ResonanceWing	Detect and couple structurally similar patterns.
CrystalWing	Extract irreducible structural cores.

MaterializationWing	Move structure into blueprint / SystemCube / Foundry space.
ValidationWing	Check replay, gate consistency, build/test/closure.
ResidueWing	Store non-integrable structure as negative evidence / deferred residue.

Chapter 12

Wing Algebra and Ophanim Topology

12.1 Compositions

Parallel

$(w_i \parallel w_j)(x) = (w_i(x), w_j(x))$ — different horizons on one space.

Sequential

$(w_j \circ w_i)(x) = w_j(w_i(x))$ — a wing works on another's condensate.

Resonant

$w_i \bowtie w_j = \text{Meet}(w_i(x), w_j(x))$ — joint invariant formation.

Conflict

$w_i \perp w_j$ — conflict is not swallowed; it is written to residue/boundary.

12.2 Wing mesh

A wing mesh is $\mathcal{W} = (V_{\mathcal{W}}, E_{\mathcal{W}})$ with $V_{\mathcal{W}} = \{w_1, \dots, w_n\}$; edges carry composition types $\{\text{parallel, sequential, resonant, conflict, defer}\}$.

12.3 Ophanim cycle

Definition 12.1 (Ophanim). An Ophanim is a closed wing cycle $\Omega = (w_1, w_2, \dots, w_k, w_1)$. It produces not mere output but repeatable self-calibration: $d(\Omega^t(x), A^*) \rightarrow 0$. A cycle is admissible only if it is replayable and gate-consistent end-to-end.

Chapter 13

Metatron Projection Kernel (L4)

13.1 Position

MPK sits not as a global sub-layer but as a *local* symmetry oracle inside one or more wings: $\text{MPK}_i \subset w_i$.

13.2 Function

Definition 13.1 (MPK output). For a small typed graph G_i ,

$$\text{MPK}(G_i) = (\text{Canon}(G_i), \text{Orb}(G_i), \text{Stab}(G_i), \text{Spec}(G_i), \partial G_i, \text{Report}_i).$$

MPK is finite, bounded, and deterministic. Its strength is not enumerating all graphs of all sizes but reliably classifying small structure windows—a topological microscope and scalpel.

Requirement 13.2 (Finite oracle, not global proof). MPK output MUST be treated as a local classification, never as a global proof. The graph catalog (13598 graphs for $n \leq 8$) MUST be bound with version, digest, and provenance before use.

Chapter 14

MERKABA Runtime Geometry (L6)

14.1 Role

MERKABA is not Coffin-Dragger; it operates the runtime geometry:

- **Scheduler:** when each wing / Ophanim runs.
- **Synchronizer:** ScorpioSync / phase windows / clocking.
- **Gatekeeper:** Monolith / Konus / thresholds.
- **Memory & Trace:** auditable convergence and emission history.

14.2 Oracle discipline

Requirement 14.1 (Oracle, not brain). An LLM or other generator is an oracle, not the authority. System intelligence arises from control structure, gates, synchronization, memory, boundary, and replay—not from the generator alone. LLM outputs may appear only as proposals and **MUST** be lowered into content-addressed structures before they become gate-relevant.

Chapter 15

Panoptic Projection (L5)

Panoptic projection is not all-sight; it is the canonical union of bounded wing projections, $\Pi_{\text{pan}} = \bigcup_i \Pi_i$. Under a declared domain profile it yields an auditable map of which regions are reachable, latent, blocked, contradictory, convergent, unstable, or emission-ready. TPC supplies the orientation logic (phase horizons, attractor maps, constraint collapse, gated emission, navigability); WingsOfSamael supplies the projection arms.

Chapter 16

HYPHAE, LPCM, SystemCube (L8 substrate)

16.1 HYPHAE

HYPHAE is host-bound, evidence-first, and non-copying. It assimilates topology, not authority or code. In the Coffin-Dragger system it is primarily Pull/Harvest infrastructure. Its production chain is:

system examples / host repo / expert spec / synthetic artifacts → observations → CodeHDAG or DomainHDAG
→ SystemCube projection → mesh-layer alignment → *D*-density scoring → contradiction-energy filtering →
LPCM condensation → SystemCube blueprint export → Workbench/Foundry materialization → parse-back /
validation / memory update.

16.2 LPCM

LPCM supplies controlled fragmentation and monotone contractive reduction of spurious degrees of freedom. In the Coffin-Dragger system, LPCM is the contraction mechanic of wing- and stack-condensation (it realizes *Contr* from definition 8.5).

16.3 SystemCube

A SystemCube is a canonical, content-addressed, evidence-bound topological blueprint for a system class: topology, mesh layers, roles, relations, constraints, test fibers, anti-patterns, Norm-Genes, StructuralCrystals, CollapsePlans, materialization profiles, compatibility rules, evidence links, and validation templates. *It is not source code.* The DiamondCube is the stricter, QSR-certified, irreducible condensation form of a SystemCube-related artifact.

Chapter 17

DiamondCube Condensation (L7→L8)

17.1 Definition

Definition 17.1 (DiamondCube). A DiamondCube is a QSR-certified, irreducible, replayable, materializable system core:

$$\text{DiamondCube}(K) = 1 \iff \text{QSR}(K) \wedge \text{Gate}(K) \wedge \text{Replay}(K) \wedge \text{Irr}(K) \wedge \text{Materializable}(K).$$

17.2 Strong irreducibility

Definition 17.2 (Strong irreducibility). $\forall K' \subsetneq K : \neg \text{QSR}(K')$. Any further reduction destroys stability, gate consistency, semantics, or materializability.

17.3 Diamondization

$$\text{Diamondize}(\mathcal{W}) = \text{Core}(\text{Fix}(\Omega(\mathcal{W}))) : \tag{17.1}$$

a DiamondCube is the irreducible core of the stable Ophanim cycles of a wing mesh.

Chapter 18

Metrics and the D -Energy Kernel

18.1 Tripolar epistemic energy

The ranking energy is the Kosmocrates tripolar product

$$D = \psi \cdot \rho \cdot \omega, \quad (18.1)$$

with ψ meaning / target fit, ρ coherence / structural stability, ω phase / time / HDAG-topological coherence. For a candidate cube C with blueprint units U_C and admissibility mass $A(u)$,

$$D(C) = \frac{1}{|U_C|} \sum_{u \in U_C} D(u) \cdot A(u).$$

18.2 Contradiction energy and the objective

$$K(C) = \sum_{(u,v) \in E_C} \text{conflict}(u, v) \cdot \text{weight}(u, v), \quad \boxed{\max_C D(C) - \lambda K(C) - \mu C(C)}$$

where $C(C)$ is avoidable complexity. Large cubes are penalized when mass does not increase coherent target utility.

18.3 Stack and cube metrics

$$\text{Density}(\text{Stack}_N) = \frac{\text{SemanticMass}(B_N)}{\text{EncodingCost}(B_N)}, \quad \text{Purity}(\text{Stack}_N) = 1 - \frac{|\text{Residue}(B_N)|}{|U(B_N)| + |\text{Residue}(B_N)|},$$

$$\text{Curv}(S_{n+1} \mid S_n) = d(\text{Canon}(S_{n+1}), \text{Canon}(\text{Embed}_n(S_n))) \leq \theta_{\text{curv}}, \quad \text{Irr}(B_N) = 1 - \frac{|\text{RemovableUnits}(B_N)|}{|U(B_N)|}.$$

In strict (diamond) mode $\text{Curv} = 0$ and $\text{Irr}(B_N) = 1$, or an explicitly justified tolerance.

18.4 Stage progress

$$\text{Progress}_n = \alpha \Delta \text{Density}_n + \beta \Delta \text{Purity}_n + \gamma \Delta \text{Irr}_n - \lambda \Delta \text{Contr}_n.$$

A stage is accepted iff $\text{Progress}_n \geq 0$ or a gate-justified exception holds.

Requirement 18.1 (No mass-for-mass-sake; FILL-001). Enrichment MUST NOT maximize feature/file/node count or mesh density alone. It MUST optimize target-compatible D -density under conflict and validation constraints. A host void is missing structure whose addition increases D -density without exceeding contradiction or complexity budgets.

Part IV

Implementation Specification (Repository Hand-off)

Chapter 19

Kosmocrates Binding Layer (KBL)

19.1 Purpose

KBL binds concrete Kosmocrates artifacts to the formal architecture: WishCube, IntegrationRun-Report, SystemCube, StructuralCrystalRecord, ResoniteMap, PSE Bridge Candidate, MaterializationReport, and ValidationClosureReport.

19.2 Binding invariant

Invariant 19.1 (Bound artifact fields). *Every bound artifact requires the tuple*

(id, type, evidence, policy, trace, replay, boundary, status).

Without these fields it MAY be observed but MUST NOT be pulled into a DiamondCube as a support unit.

Chapter 20

Algorithms

20.1 Attractor Stack Closure run

Listing 20.1: Attractor Stack Closure Run (ASCC)

```
1 input: S_n, corpus_delta, run_descriptor
2 assert gate(S_n)
3 E      = embed(S_n)                # canonical stage embedding
4 Delta  = collect_and_gate(corpus_delta) # gate-checked new material
5 A      = support_preserving_accretion(E, Delta)
6 C      = contractive_consolidation(A) # LPCM contraction
7 R      = residue_report(E, A, C)    # explicit residue
8 S_next = recrystallize(C, R)
9 Cert   = certify_embedding(S_n, S_next)
10 Stack  = append(S_n, S_next, Cert)
11 Bundle = fold(Stack)              # content-addressed B_N
12 if qsr_gate(Bundle):
13     emit DiamondCandidate(Bundle)
14 else:
15     emit DeferredStackReport(Bundle, R)
```

20.2 Recursive assimilation stage (CDK inner cycle)

Listing 20.2: Recursive Assimilation Stage

```
1 input: previous stage S_n, new material Delta_n, target attractor A_star, RD
2 assert replay_ok(S_n)
3 S0    = purge(S_n, RD)            # exkilibrate foreign mass
4 Sub    = seed(S0, RD)             # expose sub-attractors
5 Feedback = []
6 for s in deterministic_order(Sub, RD):
7     Feedback.append(probe(s, RD)) # validate seeds
8 Pulled   = pull(Feedback + Delta_n, RD) # draw toward attractor
9 Embedded = canonical_embed(S_n, Pulled, RD)
10 Compressed = compress(Embedded, RD) # Cen / condense
11 Candidate = certify_stage(Compressed, A_star, RD)
12 if qsr_stage(Candidate):
13     return Accepted(Candidate)
14 else:
15     return ResidueReport(Candidate)
```

20.3 Diamondization run

Listing 20.3: Diamondization over a wing mesh

```
1 input: wing_mesh W, run_descriptor RD
2 cycles = enumerate_ophanim(W, RD)           # closed wing cycles
3 stable = [c for c in cycles if replay(c) and roundtrip_ok(c, RD)]
4 fixcore = core(fixpoint(stable, RD))        # Core(Fix(Omega(W)))
5 cube = press_diamond(fixcore, RD)
6 assert qsr(cube) and gate(cube) and replay(cube) \
7       and irreducible(cube) and materializable(cube)
8 emit DiamondCubeCandidate(cube)
```

Requirement 20.1 (Determinism). All id-relevant objects MUST derive from canonical serializations. LLM outputs may appear only as proposals and MUST be lowered into content-addressed structures before gate relevance.

Chapter 21

Data Models and JSON Schemas

All digests are content-addressed canonical hashes. Q16 denotes a fixed-point rational in $[0, 1]$ at 16 fractional bits. `status` fields use the closed enum `{ pending, pass, warn, reject, defer }`.

Listing 21.1: AttractorStack

```
1 {
2   "stack_id": "digest",
3   "stage_ids": ["digest"],
4   "embedding_certificates": ["digest"],
5   "fold_bundle_id": "digest",
6   "qsr_status": "pending|pass|reject|defer"
7 }
```

Listing 21.2: StageEmbeddingCertificate

```
1 {
2   "certificate_id": "digest",
3   "from_stage": "digest",
4   "to_stage": "digest",
5   "canonical_equivalence": true,
6   "semantic_equivalence": true,
7   "residue_report_id": "digest",
8   "evidence_bundle_id": "digest"
9 }
```

Listing 21.3: StackClosureReport

```
1 {
2   "closure_id": "digest",
3   "stack_id": "digest",
4   "fold_bundle_id": "digest",
5   "density": "Q16", "purity": "Q16", "irreducibility": "Q16",
6   "curvature": "Q16", "contradiction_energy": "Q16",
7   "gate_result": "pass|warn|reject"
8 }
```

Listing 21.4: WingSpec

```
1 {
2   "wing_id": "digest",
3   "kind": "scout|resonance|crystal|materialization|validation|residue",
4   "scope": "domain-id",
5   "operators": ["probe", "orbit", "harvest", "condense", "anchor"],
6   "mpk_profile": "optional-profile-id",
7 }
```

```
7  "gate_profile": "policy-id",
8  "trace_root": "digest"
9  }
```

Listing 21.5: OphanimCycle

```
1  {
2  "cycle_id": "digest",
3  "wings": ["wing-digest"],
4  "composition_edges": ["parallel|sequential|resonant|conflict|defer"],
5  "roundtrip_status": "pass|warn|reject|defer",
6  "qsr_relevance": "Q16",
7  "trace_root": "digest"
8  }
```

Listing 21.6: DiamondCubeCandidate

```
1  {
2  "diamond_id": "digest",
3  "source_stack_id": "digest",
4  "systemcube_id": "digest|null",
5  "qsr_certificate_id": "digest",
6  "support_units": ["digest"],
7  "residue_report_id": "digest",
8  "materialization_profile": "report_only|dry_run|operator_approved",
9  "pse_bridge_candidate_id": "digest|null"
10 }
```

Chapter 22

Rust Type and Trait Surface

The following is a normative trait surface. Field digests are `ContentId` newtypes over the canonical hash; all fallible operations return `Result<_, GateError>` so that failure is fail-closed.

Listing 22.1: Core types (kosmo-cdk-core)

```
1 pub struct ContentId(pub [u8; 32]);
2 pub enum Status { Pending, Pass, Warn, Reject, Defer }
3 pub struct Q16(pub u32); // fixed-point in [0,1]
4
5 pub struct Stage {
6     pub attractor: ContentId,
7     pub units: Vec<ContentId>,
8     pub evidence: ContentId,
9     pub gate_status: Status,
10    pub trace_root: ContentId,
11    pub residue: ContentId,
12    pub metrics: StageMetrics,
13    pub certificates: Vec<ContentId>,
14 }
15 pub struct StageMetrics {
16     pub density: Q16, pub purity: Q16,
17     pub irreducibility: Q16, pub curvature: Q16,
18     pub contradiction_energy: Q16,
19 }
20 pub struct AttractorStack {
21     pub stack_id: ContentId,
22     pub stages: Vec<ContentId>,
23     pub embedding_certs: Vec<ContentId>,
24     pub fold_bundle: Option<ContentId>,
25     pub qsr_status: Status,
26 }
27
28 pub trait Canonical { fn canon(&self) → ContentId; }
29 pub trait Replayable { fn replay_ok(&self, rd: &RunDescriptor) → bool; }
30
31 pub trait Qsr {
32     fn qsr_stage(&self, prev: Option<&Stage>) → Result<bool, GateError>;
33     fn qsr_stack(&self) → Result<bool, GateError>;
34 }
```

Listing 22.2: Wing and Ophanim (kosmo-wings)

```
1 pub enum WingKind { Scout, Resonance, Crystal, Materialization, Validation, Residue }
2
3 pub trait Wing {
```

```

4   fn kind(&self) → WingKind;
5   // w = anchor . condense . harvest . orbit . probe
6   fn apply(&self, x: &Structure, rd: &RunDescriptor)
7       → Result<WingOutput, GateError>;
8   // INV: dim(out) ≤ dim(x) OR dim_relevant(out) < dim_relevant(x)
9   fn condenses(&self, x: &Structure, out: &WingOutput) → bool;
10  }
11
12  pub enum Composition { Parallel, Sequential, Resonant, Conflict, Defer }
13  pub struct WingMesh { pub wings: Vec<ContentId>, pub edges: Vec<(usize,usize,Composition)> }
14
15  pub trait Ophanim {
16      fn cycle(&self) → &[ContentId];
17      fn roundtrip(&self, rd: &RunDescriptor) → Result<Status, GateError>;
18      // self-calibration: d(Omega^t(x), A_star) → 0
19      fn converges(&self, rd: &RunDescriptor) → bool;
20  }

```

Listing 22.3: Coffin-Dragger run and diamond (kosmo-coffindragger)

```

1  pub trait CdkRun {
2      fn pull(&self, s: &Stage, rd: &RunDescriptor) → Result<Stage, GateError>;
3      fn purge(&self, s: &Stage, rd: &RunDescriptor) → Result<Stage, GateError>;
4      fn seed(&self, s: &Stage, rd: &RunDescriptor) → Result<Vec<Stage>, GateError>;
5      fn probe(&self, s: &Stage, rd: &RunDescriptor) → Result<Evidence, GateError>;
6      fn centralize(&self, fb: &[Evidence], rd: &RunDescriptor)
7          → Result<Stage, GateError>;
8  }
9  pub trait StackClosure {
10     fn embed(&self, s: &Stage) → Result<(Stage, ContentId), GateError>;
11     fn accrete(&self, e: &Stage, d: &Delta) → Result<Stage, GateError>;
12     fn contract(&self, s: &Stage) → Result<Stage, GateError>; // LPCM
13     fn fold(&self, stack: &AttractorStack) → Result<FoldBundle, GateError>;
14 }
15 pub trait Diamondize {
16     fn press(&self, core: &FoldBundle, rd: &RunDescriptor)
17         → Result<DiamondCubeCandidate, GateError>;
18     // requires QSR & Gate & Replay & Irreducible & Materializable
19 }

```

Chapter 23

Module / Crate Layout

Listing 23.1: Provisional crate skeleton; the agent validates against the real repo

```
1 crates/kosmo-cdk-core
2     types.rs    // AttractorStack, Stage, Delta, Certs
3     qsr.rs      // QSR predicates and certificates
4     metrics.rs  // density, purity, irreducibility, curvature, D-energy
5     gates.rs    // CDK / ASCC / KBL gates
6 crates/kosmo-wings
7     wing.rs     // Wing trait and operator path
8     mesh.rs     // WingMesh and compositions
9     ophanim.rs  // closed wing cycles
10    mpk_bridge.rs // local symmetry oracle binding (metatron-scan)
11 crates/kosmo-coffindrager
12    run.rs       // Pull / Purge / Seed / Probe / Cen loop
13    binding.rs   // KBL from Kosmocrates artifacts
14    stack.rs     // ASCC fold / closure
15    diamond.rs   // DiamondCubeCandidate generation
16 tools/kosmo-cdk
17    main.rs      // bind, stack, close, diamond, explain
```

Chapter 24

API and CLI

24.1 CLI verbs

Verb	Behavior
<code>bind</code>	Bind Kosmocrates artifacts into KBL units; enforce invariant 19.1 ; emit bound-unit ledger.
<code>stack</code>	Build / extend an AttractorStack; emit StageEmbeddingCertificates.
<code>close</code>	Run ASCC fold; emit StackClosureReport and FoldBundle.
<code>diamond</code>	Attempt diamondization of a closed stack / wing mesh; emit Diamond-CubeCandidate or DeferredStackReport.
<code>explain</code>	Render the trace / residue / gate decisions for any emitted artifact (audit surface).

24.2 REST endpoints (mirror of CLI)

POST `/cdk/bind`, POST `/cdk/stack`, POST `/cdk/close`, POST `/cdk/diamond`, GET `/cdk/explain/{id}`.
Every request carries a `RunDescriptor`; every response carries a `trace_root` and a `gate_result`.

Chapter 25

Gate and Safety Discipline

25.1 Mandatory gate cascade

The gates run as a fail-closed cascade; each must pass before the next is evaluated.

Gate	Checks
ScopeGate	Request is within the declared domain profile / authority.
EvidenceGate	Every support unit is evidence-bound.
BoundaryGate	Non-integrable mass is exkalibrated, not absorbed.
CanonicalizationGate	Inputs reduce to deterministic canonical form.
WingGate	Wing output condenses (invariant 11.2).
MPKProjectionGate	MPK used as local oracle with versioned catalog.
OphanimRoundtripGate	Cycle is replayable and roundtrip-stable.
StackClosureGate	Fold preserves canonical identity of last stage.
QSRGate	Stage-/stack-QSR predicates hold.
MaterializationSafetyGate	Materialization profile + policy + (if required) expert signoff.
ResidueVisibilityGate	All removed mass appears in residue/boundary.

25.2 Non-negotiable rule

Requirement 25.1 (Scores never override gates).

$$\text{Score}(x) > \text{Score}(y) \not\Rightarrow \text{Pass}(x).$$

Scores prioritize; gates decide. For non-code or safety-relevant domains, materialization SHOULD require domain-expert signoff before operational use.

Chapter 26

Negative Tests and Conformance

26.1 Negative tests (must be detected and fail-closed)

1. A successor stage deletes bearing units without a residue entry.
2. An embedding changes the canonical structure of the predecessor.
3. A stage only copies its predecessor instead of assimilating it.
4. Support grows but density falls; or purity falls without an explicit gate.
5. Contraction removes semantically bearing units.
6. A stack is folded though gate or replay is missing.
7. An ACDC roundtrip yields a different attractor without an explained delta gain.
8. A wing integrates structure without evidence.
9. MPK output is treated as a global proof instead of a local oracle.
10. An Ophanim cycle is not replayable.
11. A DiamondCube is produced without a QSR certificate.
12. Residue disappears from reports.
13. Materialization writes without operator-approved policy.
14. A score overrides a gate.
15. An LLM oracle is treated as authority rather than proposal source.
16. Graph-catalog / MPK data used without version / digest / provenance.

26.2 Conformance classes

Class	Requirement
ASCC-0	Terminology cleaned; stage objects defined.
ASCC-1	Every successor stage carries an embedding certificate.
ASCC-2	Support accretion and residue reports implemented.
ASCC-3	Contractive consolidation is measurable and replayable.
ASCC-4	Stack closure emits a content-addressed FoldBundle.

ASCC-5	ACDC roundtrip checked via stack closure.
ASCC-6	Diamondization emits a QSR-certified DiamondCube.
CDK-W	Wing/Ophanim layer realized with the full gate cascade.
KBL-1	Kosmocrates artifacts bound under invariant 19.1 .

26.3 Definition of Done

The implementation is done when: all crates build and test green; **bind**, **stack**, **close**, **diamond**, **explain** run end-to-end on a report-only profile; every negative test above is covered by a failing-closed test; conformance classes ASCC-0..6, CDK-W, KBL-1 are demonstrated; and a sample DiamondCubeCandidate is emitted with a complete QSR certificate, residue report, and replayable trace.

Chapter 27

Repository-Agent Work Plan

27.1 Phase 1 — Inventory

Scan the repository and tabulate: crates/modules for HYPHAE, LPCM, SystemCube, WishCube, Materialization, PSE Bridge, Metatron/graph scan, Traverse/Cognition/Projection, PhaseMatrix/Hivemind; existing reports (IntegrationRunReport, MaterializationReport, ValidationClosureReport, StructuralCrystalRecord, ResoniteMap); existing Gate/Policy/RunDescriptor/ Trace/Replay types; graph-catalog / MPK data sources; existing CLI commands and tests.

27.2 Phase 2 — Mapping

Formal entity	Repository mapping
Wing	Existing adapter/arm/SourceCube/projection worker, or a new trait.
Ophanim	Existing swarm/cycle/scheduler, or a new WingCycle.
MPK	Existing graph-scan/catalog/symmetry component, or a new submodule.
DiamondCube	Extension/wrapper of SystemCube plus QSR certificate.
QSR certificate	Existing validation/closure certificate, or a new report type.
KBL	Binding layer between existing reports and CDK stages.

27.3 Phase 3 — Specification

Produce a repository-native specification: module layout, Rust types, trait signatures, JSON schemas, CLI commands, gate matrix, negative tests, report-only MVP, dry-run plan, and a later operator-approved path. *No direct patch without a specification mapping.*

27.4 Phase 4 — Implementation tasks

Only after Phase 3 is complete, decompose into implementation tickets for separate coding agents, ordered C0 (passive report-only cube export) first.

27.5 Success criteria

Repository inventory; formalism $\leftrightarrow$ code mapping table; repo-native specification; module/crate design; data models and JSON schemas; gate and negative-test matrix; report-only MVP plan;

and a list of open questions resolvable only by code inspection.

Chapter 28

Direct Hand-off Prompt for Claude Code

Listing 28.1: Hand-off prompt

```
1 You are a repository-grounded architecture and specification agent for the
2 Kosmocrates repository. Your task is NOT to implement immediately. Derive a
3 repository-specific specification from this closed Coffin-Dragger / Wings of
4 Samael master specification, then implement in staged, gated tickets.
5
6 Operate in four phases:
7 1. Inventory the repository: crates, modules, reports, gate types, run
8   descriptors, trace/replay objects, materialization paths, and the
9   HYPHAE/LPCM/SystemCube/WishCube/PSE/Metatron/TPC components.
10 2. Map the formal entities in this document to existing code, or mark them
11   explicitly as new required abstractions.
12 3. Produce a repository-native formal specification for
13   kosmo-cdk-core / kosmo-wings / kosmo-coffindragger (or nearest equivalents).
14 4. Only after the specification is complete, create implementation tasks for
15   separate coding agents, starting with a report-only MVP (C0).
16
17 Hard constraints:
18 - No direct host mutation in the specification phase.
19 - Scores rank; gates decide.
20 - Every support unit must be evidence-bound and replay-aware.
21 - Every non-integrable unit must become visible residue.
22 - MPK is a finite local oracle, not a global proof engine.
23 - CoffinDragger is the attractor-condensation architecture, not a scraper and
24   not an autonomous authority.
25 - Wings of Samael is the codename for the Radial Arm Projection Layer.
26 - A DiamondCube requires QSR, Gate, Replay, Irreducibility, and a
27   Materialization-path evidence record.
28 - All id-relevant objects derive from canonical serializations; LLM output is
29   proposal-only until lowered into content-addressed structures.
30
31 Deliverables: repository inventory report; formal mapping table; architecture
32 specification; data-model schemas; gate and negative-test matrix; report-only
33 MVP plan; implementation task decomposition.
```

Closing

Closing Formula

The consolidated overall formula is

$$\text{DiamondCube} = \text{Diamondize}\left(\text{Core}\left(\text{Fix}\left(\Omega(\text{WingsOfSamael}(\partial\text{QSR}))\right)\right)\right),$$

and, in Kosmocrates terms,

$$\text{DiamondCubeCandidate} = \text{KBL}(\text{ASCC}(\text{CDK}(\text{HYPHAE}, \text{LPCM}, \text{MPK}, \text{TPC}, \text{MERKABA}))).$$

This is not an implementation claim; it is the closed target frame for the repository-native specification and its staged, gated realization.

Appendix A

Codename and Formal-Name Mapping

Codename / picture	Formal role
Wings of Samael	Radial Arm Projection Layer.
Gravastar axis	QSR-stable fixed-axis / boundary invariance.
Ophanim	Closed wing cycles / roundtrip self-calibration.
Metatron cube	Finite graph/symmetry catalog and local MPK oracle.
Disco ball / mirror	Orbit completion and panoptic projection.
Snowball	Monotone support accretion under rising density.
Worm / caterpillar	Peristaltic stage motion: Front, Anchor, PullBody, Compress.
Quantum matryoshka	Canonical nesting without raw redundancy.
Wormhole	Identification / remapping relation (not traversal).
Coffin-Dragger	Global attractor-condensation architecture.
DiamondCube	QSR-certified, irreducible, materializable system core.

Appendix B

Terminology Cleanup

Informal image	Formal replacement
carrying growth	support-preserving accretion
segment-wise advance	recursive stage embedding
nested layers	canonical stage embedding
condensation without loss	contractive consolidation
pulling onto one point	fixpoint-convergent recrystallization

Appendix C

Source Lineage

This closed specification folds the following source stages (codenames as on file). Each is treated as a stage S_k canonically embedded into this document per definition 8.1; material not carried forward is residue, not deletion.

- PSP / ANL-SRG master (autoreflexive null-lens, self-adjoint reflection boundary).
- Quantenstarre & Exkalibration master (QSR kernel, Σ tuple, exkalibration family).
- Kaskaden-Tunnel model (dual split/mirror/inversion, tunnel boundary).
- Domain-Neutral Crystal (DNK) (non-singular condensation, hypercube compression).
- PSP-ANL Metatron Projection Kernel v1.0 (finite graph oracle, $n \leq 8$ catalog).
- HYPHAE master & SystemCube extension (host-bound accretion, $D = \psi\rho\omega$, FILL-001).
- Recursive Assimilation Stack (ACDC/CDK) v1.1; Monotone Recursive Assimilation Calculus v1.2.
- Attractor Stack Closure Calculus (ASCC) v1.3.
- Coffin-Dragger Master Handoff Package v0.9 (layer model, wings, gate matrix, agent plan).